Homework 09

Eric Chen

All code is in Python (scikit-learn). Question 1 is answered below. The Question 2 skill has to be written by hand, so it is not included here.

Question 1: Choosing an SVM kernel

Setup. The response is `chd` (n = 462; 160 cases, so the majority-class error is 0.346). The nine predictors are `sbp`, `tobacco`, `ldl`, `adiposity`, `famhist` (1 = Absent, 2 = Present), `typea`, `obesity`, `alcohol` and `age`. I split the observations into five folds myself, stratified by `chd` with seed 432 (fold sizes 93, 93, 92, 92, 92, each with 32 cases), and used the same folds for every candidate. In each fold a `StandardScaler` is fit on the four fitting folds only and then applied to the held-out fold.

Candidates (20 in total).

- Linear: C in {0.01, 0.1, 1} (3 settings).
- Polynomial, degrees 2, 3 and 4: C in {0.01, 0.1, 1} for each degree (9 settings). Held fixed: kernel $(\gamma\langle x, x'\rangle + r)^d$ with $\gamma = 1/p = 1/9$ and $r = 1$ (`coef0`).
- Radial: C in {1, 10} crossed with γ in {0.003, 0.01, 0.03, 0.1} (8 settings).

All fits use scikit-learn's `SVC` defaults otherwise: no class weights, and prediction by the sign of the decision function.

```
rng = np.random.default_rng(432)
fold = np.empty(n, dtype=int)
for cls in np.unique(y):                        # stratified 5 folds, built by hand
    idx = rng.permutation(np.flatnonzero(y == cls))
    fold[idx] = np.arange(idx.size) % 5

for name, params in candidates:
    errs = []
    for k in range(5):
        tr, te = fold != k, fold == k
        scaler = StandardScaler().fit(X[tr])  # fitting observations only
        model = SVC(**params).fit(scaler.transform(X[tr]), y[tr])
        errs.append(np.mean(model.predict(scaler.transform(X[te])) != y[te]))
```

Results. Mean 5-fold CV misclassification rate; SE = SD of the five fold errors / √5.

Kernel	C	γ	CV error	SE	Fold errors
linear	0.01		0.296	0.008	0.290, 0.323, 0.283, 0.304, 0.283
linear	**0.1**		**0.281**	**0.014**	**0.323, 0.280, 0.239, 0.272, 0.293**
linear	1		0.281	0.014	0.323, 0.280, 0.239, 0.272, 0.293
polynomial (d=2)	0.01		0.340	0.003	0.333, 0.333, 0.348, 0.348, 0.337
polynomial (d=2)	0.1		0.279	0.009	0.258, 0.290, 0.261, 0.283, 0.304
polynomial (d=2)	1		0.271	0.015	0.247, 0.312, 0.228, 0.283, 0.283

Kernel	C	γ	CV error	SE	Fold errors
polynomial (d=3)	0.01		0.327	0.007	0.301, 0.333, 0.337, 0.326, 0.337
polynomial (d=3)	0.1		0.268	0.012	0.247, 0.269, 0.239, 0.304, 0.283
polynomial (d=3)	1		0.288	0.008	0.269, 0.290, 0.272, 0.293, 0.315
polynomial (d=4)	0.01		0.290	0.004	0.280, 0.301, 0.283, 0.293, 0.293
polynomial (d=4)	0.1		0.268	0.011	0.237, 0.280, 0.250, 0.293, 0.283
polynomial (d=4)	1		0.301	0.014	0.301, 0.312, 0.272, 0.272, 0.348
radial	1	0.003	0.327	0.004	0.323, 0.323, 0.337, 0.315, 0.337
radial	1	0.01	0.275	0.008	0.269, 0.301, 0.250, 0.283, 0.272
radial	1	0.03	0.271	0.007	0.290, 0.258, 0.250, 0.283, 0.272
radial	1	0.1	0.290	0.011	0.258, 0.301, 0.272, 0.304, 0.315
radial	10	0.003	0.266	0.017	0.312, 0.280, 0.217, 0.239, 0.283
radial	10	0.01	0.271	0.006	0.269, 0.269, 0.250, 0.283, 0.283
radial	10	0.03	0.279	0.013	0.258, 0.301, 0.250, 0.272, 0.315
radial	10	0.1	0.305	0.011	0.290, 0.301, 0.293, 0.293, 0.348

The preferred setting is in bold.

Preferred model: linear kernel, C = 0.1. The lowest mean CV error is 0.266, from the radial kernel with C = 10 and γ = 0.003. It also has the largest SE (0.017), and its γ sits at the lower edge of the grid. Nearly every well-tuned candidate falls between 0.266 and 0.29: the best of each kernel are radial 0.266, polynomial d = 3 and d = 4 both 0.268, quadratic 0.271 and linear 0.281. These gaps are smaller than one standard error, so with 462 observations and five folds the data cannot tell the kernels apart. Under the one-standard-error rule (0.266 + 0.017 = 0.283), the linear SVM with C = 0.1 (0.281) qualifies. It is the simplest candidate: it has one tuning parameter instead of two or three, and its coefficients are interpretable. C = 0.1 and C = 1 give identical fold errors, so the choice is stable in C. The nonlinear kernels cut the CV error by at most about 1.5 percentage points, which does not justify their extra flexibility and tuning. Everything clearly worse is underfit: C = 0.01 for the polynomials and γ = 0.003 with C = 1 for the radial kernel are close to the majority-class error of 0.346.